

Build Your Own Redis

A Zero-to-Interview Ready Guide

Core Server + Replication in Python

What this guide covers

A from-scratch implementation of a Redis-compatible server, focusing on the high-signal sections that interviewers actually ask about: TCP networking, the RESP protocol, concurrency, key expiry, and master-replica replication.

Language: Python 3.10+

Sections built: Core (PING, ECHO, SET/GET, Expiry, Concurrency)
+ Replication (Master-Replica handshake, propagation)

Estimated time: 10–15 hours

Prerequisites: Python basics, command line, Git

Contents

1	Introduction	2
1.1	What is Redis?	2
1.2	What are we actually building?	2
1.3	Why this project for interviews?	2
2	Fundamental Concepts (Read This First)	3
2.1	TCP sockets in one page	3
2.2	The RESP protocol	3
2.3	Concurrency models	4
2.4	Replication basics	4
3	Setup: Getting Started	5
3.1	Sign up for CodeCrafters	5
3.2	Clone and prepare	5
3.3	Install the real Redis CLI for testing	5
4	Stage 1: Bind to a Port	6
4.1	Goal	6
4.2	Code	6
4.3	Code explanation	6
4.4	Test it	6
5	Stage 2: Respond to PING	7
5.1	Goal	7
5.2	Code	7
5.3	Code explanation	7
6	Stage 3 & 4: Multiple PINGs and Concurrent Clients	8
6.1	Goal	8
6.2	The problem with the previous version	8
6.3	Code	8
6.4	Code explanation	9
7	Stage 5: Implement ECHO — The RESP Parser	10
7.1	Goal	10
7.2	Building a RESP parser	10
7.3	Code explanation	11
8	Stage 6: SET and GET — Adding State	12
8.1	Goal	12
8.2	The shared-state problem	12
8.3	Code explanation	13
9	Stage 7: Key Expiry	14
9.1	Goal	14
9.2	Strategy	14
9.3	Code (only the changed parts)	14
9.4	Code explanation	15
9.5	Test it	15

10 Replication: Building a Distributed System	16
10.1 The big picture	16
10.2 Replication protocol overview	16
10.3 Adding INFO and master/replica role	16
10.4 Code explanation	17
11 Replication: Sending the Handshake (Replica Side)	18
11.1 Goal	18
11.2 Code explanation	19
12 Replication: Master Side (Accepting Replicas & Propagating)	20
12.1 Master-side changes	20
12.2 Code explanation	21
13 End-to-End Testing	22
13.1 Testing locally	22
13.2 Pushing to CodeCrafters	22
14 Interview Prep: Questions You Will Be Asked	23
14.1 System design questions	23
14.2 Concepts to know cold	23
14.3 Resume bullets	24
15 Appendix: Common Pitfalls	25
15.1 Where to go next	25
15.2 Further reading	25

1 Introduction

1.1 What is Redis?

Redis (REmote DIctionary Server) is an in-memory data structure store. At its simplest, think of it as a giant dictionary that lives in RAM and can be accessed over the network. Real-world systems use it as:

- **Cache** — store database query results so you don't hit the DB repeatedly
- **Session store** — keep user login sessions in a fast-access place
- **Message broker** — pub/sub between services
- **Rate limiter** — track API request counts per user
- **Leaderboard** — sorted sets for real-time rankings (gaming, social)

1.2 What are we actually building?

A TCP server that speaks the Redis protocol. When a client (the official `redis-cli`, or any Redis library in any language) connects to our server on port 6379 and sends `PING`, our server replies `PONG`. When the client sends `SET foo bar`, we store it. `GET foo` returns `bar`. This is real protocol compatibility, not a toy.

1.3 Why this project for interviews?

Interview Signal

This project demonstrates four things interviewers heavily probe:

1. **Network programming** — raw TCP sockets, not HTTP frameworks
2. **Protocol design** — parsing a binary-ish wire format (RESP)
3. **Concurrency** — handling many clients at once
4. **Distributed systems** — replication, consistency, ACKs

When asked "how does Redis work?" or "design a cache", you'll answer from experience instead of memorization.

2 Fundamental Concepts (Read This First)

Before writing code, you must understand four concepts. Skipping these will leave you confused later.

2.1 TCP sockets in one page

A **socket** is the operating system's abstraction for a network connection. The flow on the server side is always:

1. Create a socket
2. **Bind** it to an address and port (e.g., 127.0.0.1:6379)
3. **Listen** for incoming connections
4. **Accept** a connection → get a new socket for that specific client
5. **Recv** bytes / **Send** bytes through that client socket
6. **Close** the socket when done

Key insight

The "listening socket" is just a doorman. Every time a client connects, you get a brand new socket dedicated to that client. You communicate with the client through *that* socket, not the listener.

2.2 The RESP protocol

RESP (REdis Serialization Protocol) is how clients and servers talk. It's a simple text protocol with five data types, each identified by a prefix byte:

Prefix	Type	Example
+	Simple String	+OK\r\n
-	Error	-ERR unknown command\r\n
:	Integer	:1000\r\n
\$	Bulk String	\$5\r\nhello\r\n
*	Array	*2\r\n\$3\r\nfoo\r\n\$3\r\nbar\r\n

Every part ends with \r\n (carriage return + line feed, often written CRLF).

A real example: when `redis-cli` sends `SET mykey hello`, the bytes on the wire are:

```

1 *3\r\n
2 $3\r\nSET\r\n
3 $5\r\nmykey\r\n
4 $5\r\nhello\r\n

```

Read it as: "an array of 3 items: bulk string of length 3 ('SET'), bulk string of length 5 ('mykey'), bulk string of length 5 ('hello')."

2.3 Concurrency models

Multiple clients want to talk to our server simultaneously. We have three choices:

1. **One thread per client** — simple, what we'll use. OS handles scheduling.
2. **Event loop (single thread)** — what real Redis uses. Much faster but harder.
3. **Async I/O** — Python's `asyncio`. Middle ground.

We'll use threads because they're conceptually simplest. In interviews, mention you *know* real Redis uses an event loop with `epoll/kqueue` for performance.

2.4 Replication basics

A **master** accepts writes. **Replicas** (slaves) copy data from the master. The flow:

1. Replica connects to master, performs a handshake
2. Master sends its current state (an RDB file)
3. Master forwards every subsequent write command to the replica
4. Replica applies each command, staying in sync

3 Setup: Getting Started

3.1 Sign up for CodeCrafters

Go to <https://codecrafters.io>, sign up, and start the Redis challenge. You'll choose Python as your language. CodeCrafters will give you:

- A starter Git repository
- Automated tests that run on `git push`
- Stage-by-stage instructions

3.2 Clone and prepare

```
1 # Clone your CodeCrafters repo (URL from their dashboard)
2 git clone https://git.codecrafters.io/<your-id>/codecrafters-redis-
  python
3 cd codecrafters-redis-python
4
5 # Project structure you'll see:
6 # app/main.py                <-- your code goes here
7 # spawn_redis_server.sh     <-- runs your server
8 # codecrafters.yml          <-- config
```

3.3 Install the real Redis CLI for testing

You need `redis-cli` (the official Redis client) to test your server.

```
1 # Ubuntu / Debian
2 sudo apt-get install redis-tools
3
4 # macOS
5 brew install redis
6
7 # Verify
8 redis-cli --version
```

Workflow

Two terminals: one runs `./spawn_redis_server.sh` (your server), the other runs `redis-cli PING` (the client). Every change: save file, restart server, test with CLI. Once happy, `git push` to run CodeCrafters' tests.

4 Stage 1: Bind to a Port

4.1 Goal

Make a TCP server that listens on port 6379 (Redis' default). It doesn't need to do anything yet — just accept connections.

4.2 Code

```
1 import socket
2
3 def main():
4     # AF_INET = IPv4, SOCK_STREAM = TCP
5     # reuse_port=True lets us restart the server quickly without
6     # "Address already in use" errors during development.
7     server_socket = socket.create_server(
8         ("localhost", 6379),
9         reuse_port=True
10    )
11
12    # Wait for a client to connect. accept() blocks here.
13    # It returns a (new_socket, client_address) tuple.
14    server_socket.accept()
15
16 if __name__ == "__main__":
17     main()
```

Listing 1: app/main.py - Bind to port

4.3 Code explanation

- `socket.create_server((host, port))` is a Python 3.8+ helper. Internally it does `socket() + bind() + listen()` in one call.
- `("localhost", 6379)` is the address. Localhost means only connections from the same machine. Port 6379 is Redis' default.
- `reuse_port=True` sets the `SO_REUSEADDR` socket option. Without it, you get an "Address already in use" error for 30–60 seconds after killing your server.
- `server_socket.accept()` blocks the program until a client connects. After `accept` returns, the program ends — but the test passes because the port was bound.

4.4 Test it

```
1 # Terminal 1
2 ./spawn_redis_server.sh
3
4 # Terminal 2 - should connect successfully
5 redis-cli PING
6 # (server will exit, but connection was made)
```


5 Stage 2: Respond to PING

5.1 Goal

When the client sends **PING**, respond with **+PONG\r\n**.

5.2 Code

```
1 import socket
2
3 def main():
4     server_socket = socket.create_server(
5         ("localhost", 6379),
6         reuse_port=True
7     )
8
9     # Wait for a client. We get back the client's socket.
10    client_socket, client_address = server_socket.accept()
11
12    # Read whatever the client sent (up to 1024 bytes)
13    # We ignore the contents for now and assume it's PING.
14    data = client_socket.recv(1024)
15
16    # Send back +PONG with the required CRLF terminator.
17    # b"..." means "bytes" (network sends bytes, not strings).
18    client_socket.send(b"+PONG\r\n")
19
20    client_socket.close()
21
22 if __name__ == "__main__":
23     main()
```

Listing 2: app/main.py - Respond to PING

5.3 Code explanation

- `accept()` returns a tuple. The first item is a **new socket** dedicated to this client.
- `recv(1024)` reads up to 1024 bytes. The `b""` indicates bytes (not strings) — this is important on networks.
- `send(b"+PONG\r\n")` writes the RESP-formatted response. The `+` prefix means "Simple String", and `\r\n` terminates the message.
- `close()` releases the OS resources for this connection.

Common pitfall

`recv()` returns bytes. `"PONG"` (a string) won't work — you need `b"PONG"` (bytes). Strings are Unicode in Python 3; the network transmits raw bytes.

6 Stage 3 & 4: Multiple PINGs and Concurrent Clients

6.1 Goal

Handle (a) multiple PINGs from the same client without disconnecting, and (b) multiple clients connecting at the same time.

6.2 The problem with the previous version

It accepts *one* connection, sends *one* reply, and exits. We need:

- A loop to keep accepting new clients (the outer loop)
- A loop per client to keep reading their commands (the inner loop)
- A way to handle multiple clients at once (threads)

6.3 Code

```
1 import socket
2 import threading
3
4 def handle_client(client_socket):
5     """Handle one client. Runs in its own thread."""
6     try:
7         while True:
8             data = client_socket.recv(1024)
9             if not data:
10                 # Empty bytes means client disconnected
11                 break
12                 # For now, reply PONG to anything.
13                 client_socket.send(b"+PONG\r\n")
14     finally:
15         client_socket.close()
16
17 def main():
18     server_socket = socket.create_server(
19         ("localhost", 6379),
20         reuse_port=True
21     )
22
23     while True:
24         # Wait for a new client
25         client_socket, _ = server_socket.accept()
26
27         # Spin up a thread to handle them.
28         # daemon=True means the thread dies when main process exits.
29         thread = threading.Thread(
30             target=handle_client,
31             args=(client_socket,),
32             daemon=True
33         )
34         thread.start()
35
36 if __name__ == "__main__":
37     main()
```

Listing 3: app/main.py - Concurrent clients

6.4 Code explanation

- **Outer** `while True` in `main()`: keep accepting new clients forever.
- **Inner** `while True` in `handle_client`: keep reading commands from *this* client.
- `if not data: break` — when a client disconnects, `recv` returns `b""`. We must check for this or we'll loop forever.
- `threading.Thread(...).start()` — creates a new OS thread for each client. The main thread immediately returns to `accept()` for the next client.

Interview Q: Why threads here?

Threads are simple but waste memory and don't scale to thousands of clients (each thread takes a few MB of stack). Real Redis uses a **single-threaded event loop** with `epoll/kqueue` to handle 100k+ connections on one CPU core. We're trading scalability for code clarity.

7 Stage 5: Implement ECHO — The RESP Parser

7.1 Goal

Implement `ECHO hello` which should return `hello`. To do this, we must actually parse the RESP protocol instead of blindly replying PONG.

7.2 Building a RESP parser

Recall: `ECHO hello` arrives as:

```
1 *2\r\n$4\r\nECHO\r\n$5\r\nhello\r\n
```

We need to break this into `["ECHO", "hello"]`.

```
1 import socket
2 import threading
3
4 def parse_resp(data: bytes):
5     """
6     Parse a RESP-encoded command and return a list of strings.
7     Example: b"*2\r\n$4\r\nECHO\r\n$5\r\nhi\r\n" -> ["ECHO", "hi"]
8     """
9     # Split on CRLF. Filter out empty trailing element.
10    parts = data.split(b"\r\n")
11    # parts looks like: [b"*2", b"$4", b"ECHO", b"$5", b"hi", b""]
12
13    if not parts or not parts[0].startswith(b"*"):
14        return []
15
16    # parts[0] is "*N" - number of array elements (not needed for
17    # parsing).
18    # We just walk through, skipping the length-prefix lines.
19    result = []
20    i = 1 # skip the *N line
21    while i < len(parts):
22        if parts[i].startswith(b"$"):
23            # Next part is the actual string
24            result.append(parts[i + 1].decode("utf-8"))
25            i += 2 # skip both length and value lines
26        else:
27            i += 1
28    return result
29
30 def encode_bulk_string(s: str) -> bytes:
31     """Convert 'hello' into the bytes b'$5\r\nhello\r\n'"""
32     return f"${len(s)}\r\n{s}\r\n".encode("utf-8")
33
34
35 def handle_client(client_socket):
36     try:
37         while True:
38             data = client_socket.recv(1024)
39             if not data:
40                 break
41
```

```

42         command = parse_resp(data)
43         if not command:
44             continue
45
46         cmd = command[0].upper()
47
48         if cmd == "PING":
49             client_socket.send(b"+PONG\r\n")
50         elif cmd == "ECHO":
51             # command[1] is the argument to echo back
52             client_socket.send(encode_bulk_string(command[1]))
53         else:
54             client_socket.send(b"-ERR unknown command\r\n")
55     finally:
56         client_socket.close()
57
58
59 def main():
60     server_socket = socket.create_server(
61         ("localhost", 6379), reuse_port=True
62     )
63     while True:
64         client_socket, _ = server_socket.accept()
65         threading.Thread(
66             target=handle_client,
67             args=(client_socket,),
68             daemon=True
69         ).start()
70
71
72 if __name__ == "__main__":
73     main()

```

Listing 4: app/main.py - RESP parser

7.3 Code explanation

- **parse_resp**: splits the incoming bytes on `\r\n`. Then it walks through, skipping every `$N` length-prefix line and grabbing the actual string that follows it.
- **encode_bulk_string**: builds the wire format for a string response. For `"hello"`, output is `$5\r\nhello\r\n`.
- **Command dispatch**: `cmd.upper()` because Redis is case-insensitive (PING and ping both work).
- **Error response**: starts with `-` per RESP spec.

Production-grade parsing

This parser assumes each `recv()` contains one complete command. In reality, TCP can split commands across multiple `recv()` calls (or batch multiple commands into one). A robust parser buffers bytes and parses incrementally. For interview discussion, mention this limitation.

8 Stage 6: SET and GET — Adding State

8.1 Goal

Implement **SET key value** and **GET key**. Now we need shared state across clients.

8.2 The shared-state problem

Each client runs in its own thread. They all need to read/write the same dictionary. If two threads write at the same time, you can corrupt the data. We use a `threading.Lock` to serialize access.

```

1 import socket
2 import threading
3
4 # Global shared state. The lock prevents race conditions.
5 data_store = {}
6 store_lock = threading.Lock()
7
8 def parse_resp(data: bytes):
9     parts = data.split(b"\r\n")
10    if not parts or not parts[0].startswith(b"*"):
11        return []
12    result = []
13    i = 1
14    while i < len(parts):
15        if parts[i].startswith(b"$"):
16            result.append(parts[i + 1].decode("utf-8"))
17            i += 2
18        else:
19            i += 1
20    return result
21
22 def encode_bulk_string(s: str) -> bytes:
23     return f"${len(s)}\r\n{s}\r\n".encode("utf-8")
24
25 def handle_client(client_socket):
26     try:
27         while True:
28             data = client_socket.recv(1024)
29             if not data:
30                 break
31             command = parse_resp(data)
32             if not command:
33                 continue
34
35             cmd = command[0].upper()
36
37             if cmd == "PING":
38                 client_socket.send(b"+PONG\r\n")
39
40             elif cmd == "ECHO":
41                 client_socket.send(encode_bulk_string(command[1]))
42
43             elif cmd == "SET":
44                 # SET key value
45                 key, value = command[1], command[2]

```

```

46         with store_lock:
47             data_store[key] = value
48             client_socket.send(b"+OK\r\n")
49
50     elif cmd == "GET":
51         # GET key
52         key = command[1]
53         with store_lock:
54             value = data_store.get(key)
55         if value is None:
56             # Null bulk string = key doesn't exist
57             client_socket.send(b"$-1\r\n")
58         else:
59             client_socket.send(encode_bulk_string(value))
60
61     else:
62         client_socket.send(b"-ERR unknown command\r\n")
63 finally:
64     client_socket.close()
65
66 def main():
67     server_socket = socket.create_server(
68         ("localhost", 6379), reuse_port=True
69     )
70     while True:
71         client_socket, _ = server_socket.accept()
72         threading.Thread(
73             target=handle_client,
74             args=(client_socket,),
75             daemon=True
76         ).start()
77
78 if __name__ == "__main__":
79     main()

```

Listing 5: Adding the data store

8.3 Code explanation

- `data_store = {}` — a module-level dict, shared across all threads.
- `store_lock = threading.Lock()` — mutex. `with store_lock:` acquires it on enter, releases on exit.
- SET response is `+OK\r\n` (simple string).
- GET on a missing key returns `$-1\r\n` — the "null bulk string", RESP's way of saying nil.

Interview Q: Is this thread-safe?

Python's GIL makes dict operations *mostly* safe, but the lock is still correct practice and necessary for compound operations (read-modify-write). In a non-GIL language (C++/Rust), you'd absolutely need a lock or a concurrent map.

9 Stage 7: Key Expiry

9.1 Goal

Support **SET key value PX milliseconds** which sets a key that auto-expires.

9.2 Strategy

Store `(value, expires_at_timestamp)` pairs instead of just values. On **GET**, check if the current time has passed `expires_at`; if so, delete the key and return nil.

This is called **passive expiry** — we expire on read instead of running a background cleaner. Real Redis does both passive and active expiry (periodic background scan).

9.3 Code (only the changed parts)

```

1 import time
2
3 # Store now holds (value, expires_at_ms_or_None)
4 # data_store["foo"] -> ("bar", 1700000000000) or ("bar", None)
5
6 elif cmd == "SET":
7     key, value = command[1], command[2]
8     expires_at = None
9
10    # Look for optional PX argument
11    # SET key value PX 5000
12    if len(command) >= 5 and command[3].upper() == "PX":
13        ms = int(command[4])
14        # time.time() is in seconds; we want ms
15        expires_at = int(time.time() * 1000) + ms
16
17    with store_lock:
18        data_store[key] = (value, expires_at)
19        client_socket.send(b"+OK\r\n")
20
21 elif cmd == "GET":
22     key = command[1]
23     with store_lock:
24         entry = data_store.get(key)
25         if entry is None:
26             client_socket.send(b"$-1\r\n")
27             continue
28
29         value, expires_at = entry
30         # Check expiry
31         if expires_at is not None and time.time() * 1000 >
           expires_at:
32             # Key expired; remove it
33             del data_store[key]
34             client_socket.send(b"$-1\r\n")
35         else:
36             client_socket.send(encode_bulk_string(value))

```

Listing 6: SET/GET with expiry

9.4 Code explanation

- `time.time()` returns seconds since epoch (a float). Multiply by 1000 for milliseconds.
- `expires_at` is the *absolute* timestamp when the key dies, not a duration. Storing absolute time is simpler than computing remaining TTL on every read.
- Passive deletion: we only clean up expired keys when someone tries to read them.

Interview Q: Active vs Passive Expiry

Passive: expire on read. Pro: simple, no background work. Con: dead keys waste memory if never accessed.

Active: background thread scans random keys, deletes expired ones. Real Redis does this every 100ms, sampling 20 keys.

Combined approach (what real Redis uses) gives both correctness and bounded memory waste.

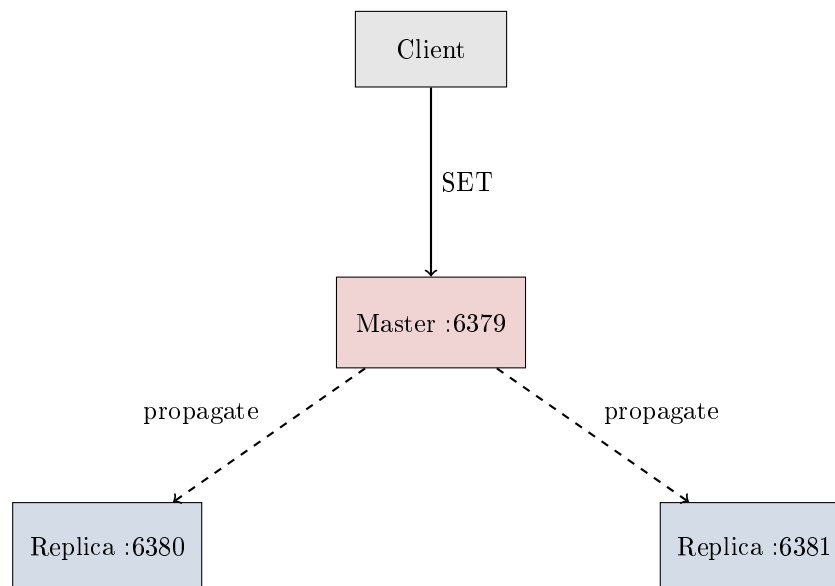
9.5 Test it

```
1 redis-cli SET foo bar PX 5000 # expires in 5 seconds
2 redis-cli GET foo             # returns "bar"
3 sleep 6
4 redis-cli GET foo             # returns (nil)
```

10 Replication: Building a Distributed System

This is the most impressive part of the project. You'll build a master that can forward writes to one or more replicas.

10.1 The big picture



The client writes to the master. The master records the write and forwards it to every replica. Replicas apply the same writes, staying in sync.

10.2 Replication protocol overview

The handshake (from replica's perspective):

1. **PING** the master — "are you alive?"
2. **REPLCONF listening-port 6380** — "I listen here"
3. **REPLCONF capa psync2** — "I support these features"
4. **PSYNC ? -1** — "I have no data; send me everything"
5. Master replies **+FULLRESYNC <replid> 0** and sends an RDB file
6. From now on, master forwards every write command

10.3 Adding INFO and master/replica role

First, accept the **-port** and **-replicaof** command-line flags, and respond to **INFO replication**.

```
1 import sys
2 import socket
3 import threading
4 import time
5
6 data_store = {}
7 store_lock = threading.Lock()
8
```

```

9  # Server configuration
10 config = {
11     "port": 6379,
12     "role": "master",          # or "slave" (Redis legacy term)
13     "master_host": None,
14     "master_port": None,
15     "master_replid": "8371b4fb1155b71f4a04d3e1bc3e18c4a990aeeb",
16     "master_repl_offset": 0,
17 }
18
19 def parse_args():
20     """Parse --port and --replicaof from sys.argv"""
21     args = sys.argv[1:]
22     i = 0
23     while i < len(args):
24         if args[i] == "--port":
25             config["port"] = int(args[i + 1])
26             i += 2
27         elif args[i] == "--replicaof":
28             # --replicaof "host port" (single quoted arg)
29             host_port = args[i + 1].split()
30             config["role"] = "slave"
31             config["master_host"] = host_port[0]
32             config["master_port"] = int(host_port[1])
33             i += 2
34         else:
35             i += 1
36
37 def handle_info():
38     """Build the INFO replication response"""
39     info_lines = [
40         f"role:{config['role']}",
41         f"master_replid:{config['master_replid']}",
42         f"master_repl_offset:{config['master_repl_offset']}",
43     ]
44     info_text = "\r\n".join(info_lines)
45     return encode_bulk_string(info_text)

```

Listing 7: Configurable server with role

10.4 Code explanation

- `config["role"]` is "master" by default. Becomes "slave" if `-replicaof` is passed.
- `master_replid` is a 40-character hex string identifying this master. Real Redis generates a random one; we hard-code for simplicity.
- `master_repl_offset` tracks how many bytes of commands we've propagated. Starts at 0.

11 Replication: Sending the Handshake (Replica Side)

11.1 Goal

When started with `-replicaof "host port"`, the replica should connect to the master and complete the 4-step handshake.

```

1 def encode_array(items):
2     """Build a RESP array from a list of strings."""
3     result = f"*{len(items)}\r\n"
4     for item in items:
5         result += f"${len(item)}\r\n{item}\r\n"
6     return result.encode("utf-8")
7
8 def perform_handshake():
9     """Connect to master and complete the replication handshake."""
10    master_sock = socket.create_connection(
11        (config["master_host"], config["master_port"])
12    )
13
14    # Step 1: PING
15    master_sock.send(encode_array(["PING"]))
16    master_sock.recv(1024) # expect +PONG
17
18    # Step 2: REPLCONF listening-port <our-port>
19    master_sock.send(encode_array([
20        "REPLCONF", "listening-port", str(config["port"])
21    ]))
22    master_sock.recv(1024) # expect +OK
23
24    # Step 3: REPLCONF capa psync2
25    master_sock.send(encode_array(["REPLCONF", "capa", "psync2"]))
26    master_sock.recv(1024) # expect +OK
27
28    # Step 4: PSYNC ? -1
29    # ? = unknown replication ID (first sync)
30    # -1 = no offset (full resync)
31    master_sock.send(encode_array(["PSYNC", "?", "-1"]))
32    response = master_sock.recv(1024)
33    # Master replies +FULLRESYNC <replid> 0\r\n
34    # then sends an RDB file: ${len}\r\n<binary bytes>
35
36    return master_sock
37
38 def main():
39     parse_args()
40
41     # If we're a replica, do handshake before listening
42     if config["role"] == "slave":
43         master_sock = perform_handshake()
44         # Spawn thread to read commands from master and apply them
45         threading.Thread(
46             target=read_from_master,
47             args=(master_sock,),
48             daemon=True
49         ).start()
50
51     server_socket = socket.create_server(

```

```
52         ("localhost", config["port"]), reuse_port=True
53     )
54     while True:
55         client_socket, _ = server_socket.accept()
56         threading.Thread(
57             target=handle_client,
58             args=(client_socket,),
59             daemon=True
60         ).start()
61
62 def read_from_master(master_sock):
63     """Continuously read commands propagated from master."""
64     while True:
65         data = master_sock.recv(4096)
66         if not data:
67             break
68         command = parse_resp(data)
69         # Apply the command locally WITHOUT sending a reply.
70         # Replicas process writes silently.
71         if command and command[0].upper() == "SET":
72             with store_lock:
73                 data_store[command[1]] = (command[2], None)
```

Listing 8: Replica handshake logic

11.2 Code explanation

- `encode_array` builds RESP arrays — which is how commands are sent over the wire.
- The replica blocks on each handshake step waiting for the master's reply.
- After handshake, `read_from_master` runs forever reading commands the master propagates.
- Crucially, replicas **do not send replies** when applying propagated commands.

12 Replication: Master Side (Accepting Replicas & Propagating)

12.1 Master-side changes

The master must:

1. Reply to REPLCONF with +OK
2. Reply to PSYNC ? -1 with +FULLRESYNC <replid> 0 and send an RDB file
3. Remember every connected replica's socket
4. On every write (**SET**), forward the command to all replicas

```

1  # Global list of replica sockets we're propagating to
2  replicas = []
3  replicas_lock = threading.Lock()
4
5  # Hardcoded empty RDB file (88 bytes, hex-encoded)
6  EMPTY_RDB_HEX = (
7      "524544495330303131fa0972656469732d76657205372e322e30"
8      "fa0a72656469732d62697473c040fa056374696d65c26d08bc65"
9      "fa08757365642d6d656dc2b0c41000fa08616f662d62617365c0"
10     "00fff06e3bfec0ff5aa2"
11 )
12 EMPTY_RDB = bytes.fromhex(EMPTY_RDB_HEX)
13
14 def handle_client(client_socket):
15     is_replica = False
16     try:
17         while True:
18             data = client_socket.recv(1024)
19             if not data:
20                 break
21             command = parse_resp(data)
22             if not command:
23                 continue
24             cmd = command[0].upper()
25
26             if cmd == "PING":
27                 client_socket.send(b"+PONG\r\n")
28
29             elif cmd == "REPLCONF":
30                 # Replica is configuring; always reply OK
31                 client_socket.send(b"+OK\r\n")
32
33             elif cmd == "PSYNC":
34                 # Send +FULLRESYNC + RDB file
35                 resp = f"+FULLRESYNC {config['master_replid']} 0\r\n"
36
37                 client_socket.send(resp.encode())
38                 # Send RDB: $<len>\r\n<binary> (note: NO trailing \r\n)
39                 client_socket.send(
40                     f"${len(EMPTY_RDB)}\r\n".encode() + EMPTY_RDB
41                 )
42                 # This client is now a replica
43                 is_replica = True
44                 with replicas_lock:
45                     replicas.append(client_socket)

```

```

45
46         elif cmd == "SET":
47             key, value = command[1], command[2]
48             with store_lock:
49                 data_store[key] = (value, None)
50                 client_socket.send(b"+OK\r\n")
51                 # Propagate to all replicas (master only)
52                 if config["role"] == "master":
53                     propagate(command)
54
55         elif cmd == "GET":
56             # ... same as before
57             pass
58
59         elif cmd == "INFO":
60             client_socket.send(handle_info())
61
62             # If this connection is a replica, don't close it after
63             # handshake - keep it open for propagation.
64         finally:
65             if not is_replica:
66                 client_socket.close()
67
68     def propagate(command):
69         """Send command to all connected replicas as a RESP array."""
70         msg = encode_array(command)
71         with replicas_lock:
72             for replica_sock in replicas:
73                 try:
74                     replica_sock.send(msg)
75                 except:
76                     pass # replica disconnected; ignore for now

```

Listing 9: Master replication logic

12.2 Code explanation

- `EMPTY_RDB` is the binary representation of an empty Redis database. CodeCrafters provides this as a known constant. We send it so the replica has a valid starting state.
- The RDB transfer format is `$<length>\r\n<bytes>` — like a bulk string but **without** the trailing `\r\n`.
- Once a client sends `PSYNC`, we mark them `is_replica = True` and add their socket to our list. We never close it.
- `propagate()` forwards every `SET` command to every connected replica.

Interview Q: What if propagation fails?

In our toy version, we silently swallow the error. Real Redis tracks per-replica offsets and supports **partial resynchronization** via `PSYNC2`: if a replica disconnects briefly and reconnects, it can resume from where it left off instead of doing a full RDB transfer.

13 End-to-End Testing

13.1 Testing locally

```
1 # Terminal 1: master on default port
2 ./spawn_redis_server.sh
3
4 # Terminal 2: replica on port 6380
5 ./spawn_redis_server.sh --port 6380 --replicaof "localhost 6379"
6
7 # Terminal 3: connect to master, write a key
8 redis-cli -p 6379 SET foo bar
9 # (OK)
10
11 # Terminal 3: read from replica - should have the same value!
12 redis-cli -p 6380 GET foo
13 # "bar"
```

If **GET foo** on the replica returns **bar**, replication works.

13.2 Pushing to CodeCrafters

```
1 git add app/main.py
2 git commit -m "Implement replication"
3 git push origin master
```

CodeCrafters will run automated tests against your code and stream pass/fail results to your terminal.

14 Interview Prep: Questions You Will Be Asked

14.1 System design questions

Q1: Walk me through what happens when a client sends SET foo bar

Answer: The bytes `*3\r\n$3\r\nSET\r\n$3\r\nfoo\r\n$3\r\nbar\r\n` arrive on the TCP socket. We `recv()` them, split on `\r\n`, walk the RESP array structure to extract `["SET", "foo", "bar"]`, acquire the data store lock, write to the dict, release the lock, send `+OK\r\n`, and if we're a master, forward the command to all replica sockets.

Q2: How would you make Redis faster?

Answer: Three angles: (1) Replace thread-per-client with a single-threaded event loop using `epoll` — avoids context-switch and lock overhead. (2) Use zero-copy I/O like `sendfile` for large bulk responses. (3) Batch responses (return multiple replies in one `send()`). Real Redis does all three.

Q3: How does replication handle consistency?

Answer: Redis replication is **asynchronous** — the master replies `+OK` to a write *before* the replicas have it. This is eventual consistency. If you need stronger guarantees, use the `WAIT n timeout` command to block until `n` replicas have ACK'd. For true strong consistency, you need Raft/Paxos — which is what Redis Cluster's failover uses internally.

Q4: What happens if a replica falls behind?

Answer: The master maintains a backlog buffer of recent commands. If the replica's offset is within the backlog, it can do a partial resync. If it's fallen too far behind, the master does a full resync: dumps the entire dataset as an RDB file, sends it, then resumes propagation.

14.2 Concepts to know cold

- **RESP protocol** — 5 types, CRLF terminators, bulk vs simple strings
- **TCP socket lifecycle** — bind, listen, accept, recv, send, close
- **Concurrency models** — threads vs event loop vs async
- **Race conditions** — why you need locks on shared state
- **Passive vs active key expiry**
- **Replication handshake** — PING, REPLCONF, PSYNC
- **RDB vs AOF** — snapshot vs append-only log; trade-offs
- **CAP theorem** — Redis is AP, not CP

14.3 Resume bullets

Suggested Resume Entry

Redis Server Implementation*Python, TCP/Sockets, Multi-threading*github.com/yourusername/redis-implementation

- Built a Redis-compatible in-memory data store from scratch supporting concurrent clients via thread-per-connection architecture.
- Implemented the RESP wire protocol parser handling 5 data types (simple strings, errors, integers, bulk strings, arrays).
- Supported core commands (PING, ECHO, GET, SET with PX expiry) and master-replica replication.
- Designed the replication handshake (REPLCONF, PSYNC) and command propagation pipeline forwarding writes from master to all connected replicas.

15 Appendix: Common Pitfalls

- **Forgetting CRLF**: every RESP message ends with `\r\n`. Easy to forget for one type and have the client hang.
- **Mixing strings and bytes**: `socket.send()` only accepts `bytes`. `recv()` returns `bytes`. Use `.encode()` / `.decode()` at the boundaries.
- **Assuming one `recv()` = one command**: TCP doesn't preserve message boundaries. A robust implementation buffers bytes and parses incrementally.
- **Closing the replica socket after handshake**: a replica connection lives forever. Don't put it in a `with` block or `try/finally` that closes it.
- **Sending replies for propagated commands**: when a replica receives a SET from the master, it must apply it *silently*. No `+OK` reply.
- **Case sensitivity**: Redis commands are case-insensitive. Always `cmd.upper()` before comparing.
- **RDB transfer format**: this is the one bulk-string-like thing that does NOT have a trailing CRLF. Just `$(len)\r\n<binary>`.

15.1 Where to go next

After core + replication, the next highest-signal additions are:

1. **RDB persistence (read)** — parse the binary dump format. Great signal for "low-level systems" roles.
2. **Transactions (MULTI/EXEC)** — demonstrates understanding of ACID.
3. **Streams** — shows pub/sub-style messaging.

15.2 Further reading

- Redis Protocol Spec: <https://redis.io/docs/latest/develop/reference/protocol-spec/>
- Redis Replication: https://redis.io/docs/latest/operate/oss_and_stack/management/replication/
- Real Redis source code: <https://github.com/redis/redis> — start with `networking.c`
- Build Your Own X (free alternative): <https://github.com/codecrafters-io/build-your-own-x>

Good luck with your interviews!